Local AI Blueprint: Organizing & Training on Family Media

A privacy-first framework for processing, deduplicating, and clustering large family media archives on Apple Silicon Macs.

1. Core Philosophy: Why Local AI?

Processing highly personal family media—spanning decades, faces of children, and private moments—requires strict privacy boundaries. Training massive Vision-Language Models (VLMs) from scratch costs millions, but **fine-tuning existing open-source models** or running a **modular machine learning pipeline** entirely on your local machine protects your data while achieving complex organizational goals.

2. Architectural Overview for 10k+ Videos

Feeding 10,000 raw video files directly into a generative Large Language Model will crash standard system memory, cause extreme thermal throttling, and fail to generate a clean structural database. Instead, an efficient pipeline uses a three-stage strategy optimized for Apple Silicon Unified Memory architectures:

- **Stage 1: Pre-Filtering & Deduplication:** Removing duplicate bitrates, compressed clips, and identical files before running neural networks.
- **Stage 2: Frame Embedding & Facial Mapping:** Extracting periodic keyframes and turning them into vector coordinates alongside specialized facial age-progression networks.
- **Stage 3: Density Clustering:** Storing vectors in a fast, serverless embedded database and letting math algorithms group related events dynamically.

3. Step-by-Step Pipeline Implementation

Step 1: Safely Deduplicate via Czkawka

Standard file hash algorithms (MD5/SHA256) miss compressed copies (e.g., WhatsApp downscales vs. original iPhone .mov files). Use **Czkawka**, a blazingly fast deduplicator written in Rust.

- **Installation:** Open Terminal and run: `brew install czkawka`
- **Rule:** Set your primary master drive as a 'Reference Folder' within the GUI. Configure the tool to *Move* rather than *Delete*, safeguarding original files while clearing out messy imports.

Step 2: Continuous Frame Embedding (The AI Eye)

Convert video chunks into high-density mathematical vectors using local embedding engines accelerated via Apple's Metal Performance Shaders (MPS):

- **Visual Scene Vectors:** Use models like **SigLIP-2** or **EVA-CLIP** via PyTorch. Route them through the Mac GPU via `torch.device('mps')`.

• **Facial Continuity:** Tracking a child growing up into an adult over decades breaks general CLIP models. Use **InsightFace (Buffalo_L)** on MPS to anchor coordinates for identical skeletal facial traits across time.

Step 3: Vector Storage & Density Clustering

Store these vector representations inside a serverless vector database like **LanceDB**. LanceDB works entirely within your local file directory, boasting zero-copy memory reads perfectly suited for the Mac's Unified Memory.

To build folders, run a density clustering algorithm such as **DBSCAN** or **HDBSCAN** via scikit-learn. Avoid K-Means, because K-Means forces you to guess the exact number of folders upfront. DBSCAN groups clusters dynamically based on vector distance, separating real 'Events' and dropping outliers into an 'Unclassified' folder.

4. Automated Organization Blueprint Script

Below is the structural Python script to parse your vector tables and automatically sort video files into logical, chronological subdirectories:

```
import os
import shutil
from datetime import datetime
import lancedb
import numpy as np
from sklearn.cluster import DBSCAN
DB_PATH = "./.lancedb_family_vault"
db = lancedb.connect(DB_PATH)
table = db.open_table("video_features")
records = table.to_pandas()
embeddings = np.stack(records["vector"].values)
timestamps = records["timestamp"].tolist()
file_paths = records["file_path"].tolist()
detected_faces = records["primary_person"].tolist()
clustering_model = DBSCAN(eps=0.35, min_samples=2, metric='cosine')
cluster_labels = clustering_model.fit_predict(embeddings)
TARGET_VAULT = "/Volumes/SecureStorage/Organized_Family_Media"
for idx, label in enumerate(cluster_labels):
    source_file = file_paths[idx]
    if not os.path.exists(source_file): continue

    folder = f"Event_Cluster_{label}" if label != -1 else "Unclassified_Moments"
    person = detected_faces[idx] if detected_faces[idx] else "Mixed_Family"
    date_str = datetime.fromtimestamp(timestamps[idx]).strftime('%Y-%m-%d')

    dest_dir = os.path.join(TARGET_VAULT, person, f"{date_str}_{folder}")
    os.makedirs(dest_dir, exist_ok=True)
    shutil.move(source_file, os.path.join(dest_dir, os.path.basename(source_file)))
```

5. Bringing In Large Video LLMs (The Final Step)

Once the data is pre-sorted into, say, 200 distinct cluster folders, you can run an actual generative vision model over just *one reference video per cluster* to name the folders automatically.

- **Framework:** Use **MLX-VLM**, an open-source architecture built specifically for Apple Silicon that leverages Native MLX, outperforming traditional PyTorch on Apple Silicon by up to 28x.
- **Model:** Run a quantified variant of **Qwen2-VL-7B** or **Gemma 3N**.
- **Prompt:** *'Analyze this clip and provide a 4-word descriptive title of the event.'* You can then programmatically rename generic directories like `Event_Cluster_42` to `1996-07_Grandpa_Birthday_Barbecue`.